DATABASE MANAGEMENT SYSTEMS · WEEK 1

Welcome to the Data Universe

Files vs DBMS, with Lab 01: Dockerized PostgreSQL and the File vs SQL Benchmark

2-hour session

Lecture + Lab + 21 quiz questions



Dr.-Ing. Chukwuka Victor Obionwu

PLAN · 2 HOURS

Session timeline

Part A · Lecture

Time	Segment	Min
0:00	Welcome & KoboKart story	5
0:05	Limits of flat files + Quiz 1	15
0:20	What a DBMS provides + Quiz 2	15
0:35	PostgreSQL ecosystem + Quiz 3	10
0:45	Break	5

Part B · Lab 01 and wrap-up

Time	Segment	Min
0:50	Lab overview & Docker setup	5
0:55	Ex 1 · Container & psql	10
1:05	Ex 2 · DDL & data import	20
1:25	Ex 3 · Index benchmark	20
1:45	Lab quiz	8
1:53	Scoring, recap, exit ticket	7

OBJECTIVES

By the end of today you can…

1. Explain three ways flat files fail under load
2. Describe what a DBMS provides: SQL, ACID, WAL, indexes
3. Spin up PostgreSQL 16 with Docker Compose and connect with psql
4. Create a table, load 100,000 orders and benchmark an indexed query

THE STORY · 0:02

You launch KoboKart, a multi-vendor e-commerce platform across West Africa.

10,000 concurrent checkout attempts per second during a flash sale. Where do the orders live?

WARM-UP · 2 MIN

Your MVP stores orders in a CSV file. What do you think breaks first?

FLAT FILES · 0:05

The MVP: orders in a CSV file

```
order_id,customer_id,merchant_id,total,status
1001,usr_881,mer_001,450.00,PAID
1002,usr_902,mer_002,1200.50,PENDING
1003,usr_881,mer_001,89.99,SHIPPED
```

orders.csv

Simple for single-threaded scripts

Risky once many users write at the same time

Human-readable, easy to start, no setup. That is why every MVP begins here.

Three failures during the flash sale

1. Race conditions and corruption

Two threads opening orders.csv in write mode overwrite each other: partial writes and corrupt rows.

2. Linear search latency, O(N)

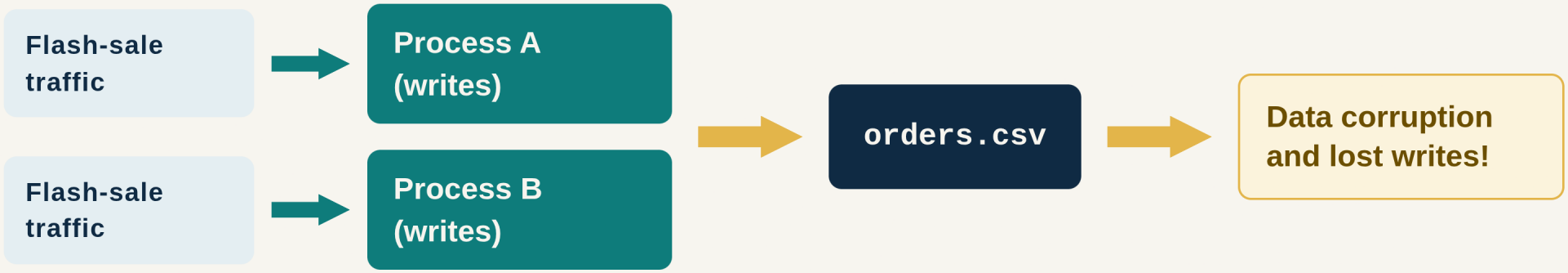
Finding all orders for usr_881 reads every line from disk into RAM and converts strings to floats on each request.

3. No integrity constraints

Nothing stops a mobile-app bug from writing a total of -450.00 or a non-existent merchant ID.

FAILURE 1 · 0:10

Two writers, one file



A file has no built-in locking. The operating system happily lets both processes write.

THINK · 1 MIN

Could you fix this in Python with your own lock? What new problems appear?

FAILURE 2 · 0:12

Linear search: every lookup reads everything



THE QUERY

Find all orders for usr_881. The teal records match, but the program must still read all 100,000+ lines to find them.

O(N)

work grows with file size

Ask · 1 min: if the file grows 10 times larger, roughly how does one lookup change?

QUIZ 1 · QUESTIONS 1–3 · 0:14

Check your understanding

QUESTION 1

Two processes write to orders.csv at the same moment. Which problem can occur?

A. An index gets corrupted

B. A race condition with lost or partial writes

C. The query optimizer picks a hash join

D. Decimal precision loss

QUESTION 2

Finding all orders for usr_881 in an unindexed CSV with N lines has what time complexity?

A. O(1)

B. O(log N)

C. O(N)

D. $O(N^2)$

QUESTION 3

A bug writes a total of -450.00 and nothing stops it. Which flat-file weakness is this?

A. Race conditions

B. Linear search latency

C. Slow disks

D. Lack of integrity constraints

Think for 30 seconds, vote A to D, then reveal the answers on the next slide.

QUIZ 1 · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: B

A race condition with lost or partial writes

Two threads in write mode overwrite each other, leaving partial writes or corrupt CSV rows.

QUESTION 2

Answer: C

O(N)

Without an index, every line must be read to find the matches, so the work grows with the file size.

QUESTION 3

Answer: D

Lack of integrity constraints

A file cannot reject a negative total or a reference to a merchant that does not exist.

QUIZ 1 · QUESTIONS 4–6 · 0:17

Check your understanding

QUESTION 4 · TRUE OR FALSE

Flat files are simple and work well for single-threaded scripts.

A. True

B. False

QUESTION 5

Which is NOT one of the three flat-file failures from the lecture?

A. Race conditions and data corruption

B. Linear search latency

C. Lack of integrity constraints

D. Too many index types to choose from

QUESTION 6

What do production systems like KoboKart need, according to the core takeaway?

A. Only faster disks

B. Bigger CSV files

C. ACID guarantees, indexing, concurrent locking, declarative querying

D. No constraints at all

Think for 30 seconds, vote, then reveal the answers on the next slide.

QUIZ 1 · ANSWERS 4–6

Answers and explanations

QUESTION 4

Answer: A (True)

Files are fine for one script writing alone. The trouble starts with concurrency, scale and integrity.

QUESTION 5

Answer: D

Too many index types

That is not a flat-file failure. A flat file has no indexes at all. Options A to C are the three failures.

QUESTION 6

Answer: C

ACID, indexing, locking, declarative querying

These four capabilities are exactly what a relational DBMS adds. Next: how.

WHAT A DBMS PROVIDES · 0:20

A DBMS manages data three ways

Specialized software that manages persistent data efficiently, safely and concurrently.

FILE PROBLEM

Linear search, O(N)

DBMS ANSWER · EFFICIENT

Indexes, O(log N)

FILE PROBLEM

No integrity rules, lost data

DBMS ANSWER · SAFE

Constraints, ACID, WAL

FILE PROBLEM

Race conditions

DBMS ANSWER · CONCURRENT

Concurrent locking

THE SAME DATA IN A DBMS · 0:23

The orders table in PostgreSQL 16

```
CREATE TABLE orders (
    order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL,
    merchant_id UUID NOT NULL,
    order_total DECIMAL(10, 2) NOT NULL
        CHECK (order_total >= 0.00),
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMP WITH TIME ZONE
        DEFAULT CURRENT_TIMESTAMP
);
```

PRIMARY KEY: each order is unique

NOT NULL: required values

CHECK (order_total >= 0.00): rejects -450.00

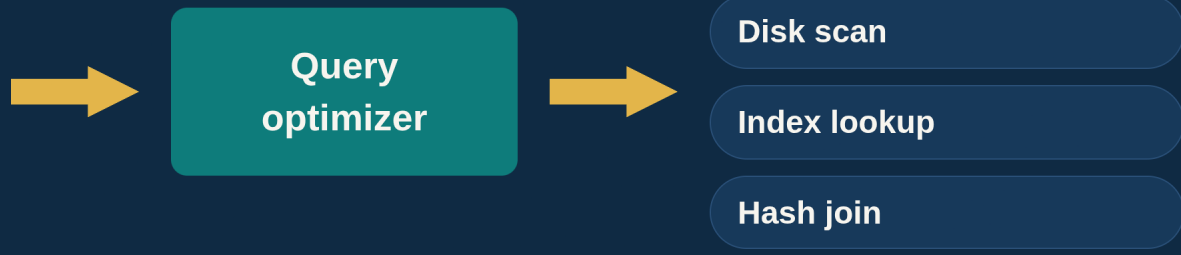
DEFAULT 'PENDING': omitted status becomes PENDING

DECLARATIVE QUERYING · 0:26

You say what. The optimizer decides how.

YOU WRITE (WHAT)

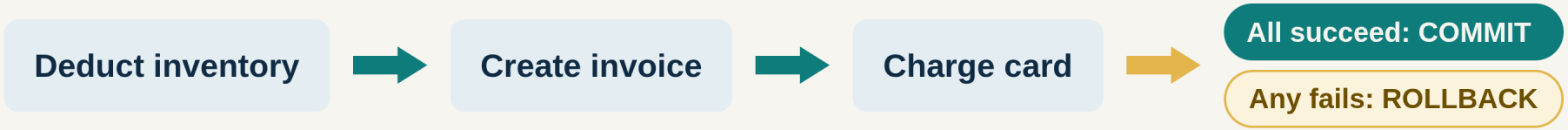
```
SELECT order_id, order_total
FROM orders
WHERE status = 'PENDING'
  AND order_total > 250.00;
```



Same SQL today and next year, even when the table grows from 1,000 to 100,000,000 rows. The plan can change; your query does not.

ACID TRANSACTIONS · 0:28

A checkout either fully happens or not at all



A

Atomicity

All steps succeed, or none take effect.

C

Consistency

Data always obeys the rules, such as constraints.

I

Isolation

Concurrent transactions do not trample each other.

D

Durability

Committed work survives crashes.

CRASH RECOVERY · 0:29

Write-Ahead Logging (WAL)

1

Write the change to the log first

2

COMMIT is confirmed once the log is safe

3

Data files are updated afterwards

!

After a crash, replay the log

Committed transactions survive sudden power loss or server failure.

CONTRAST · 1 MIN

What happens to a half-written line in orders.csv after a power cut?

INDEXES · 0:30

From O(N) to O(log N)

```
-- Indexing customer lookups: search cost drops from O(N) to O(log N)
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

Rows (N)	Full scan	Index, about $\log_2 N$
1,000	1,000	about 10
100,000	100,000	about 17
1,000,000	1,000,000	about 20

10× more rows adds only about 3 steps to an index lookup.

Step counts are an illustration of the logarithm. Real PostgreSQL B-Trees read a few disk pages, with the same idea.

QUIZ 2 · QUESTIONS 1–3 · 0:32

Check your understanding

QUESTION 1

What does “declarative querying” mean in SQL?

A. You write the exact steps to read the disk

B. You state what data you need; the optimizer decides how

C. You must name the index to use

D. Queries are only valid on one table

QUESTION 2

Inventory is deducted but the card charge fails, so everything is undone. Which ACID property is this?

A. Durability

B. Isolation

C. Atomicity

D. Consistency

QUESTION 3

What does Write-Ahead Logging (WAL) ensure?

A. Every query runs in O(1)

B. CSV files synchronize themselves

C. Indexes rebuild every day

D. Committed transactions survive sudden power loss

Think for 30 seconds, vote A to D, then reveal the answers on the next slide.

QUIZ 2 · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: B

State what, let the optimizer decide how

The optimizer picks disk scans, index lookups or hash joins for you.

QUESTION 2

Answer: C

Atomicity

A transaction either succeeds as a whole or rolls back cleanly. No half-finished checkout.

QUESTION 3

Answer: D

Committed work survives power loss

The log is written first, so after a crash the database can recover committed transactions.

QUIZ 2 · QUESTIONS 4–6 · 0:34

Check your understanding

QUESTION 4

CREATE INDEX idx_orders_customer ON orders(customer_id) changes customer lookups from…

A. O(log N) to O(N)

B. O(N) to O(log N)

C. $O(N^2)$ to O(N)

D. O(1) to O(N)

QUESTION 5

Which clause in the orders table rejects a total of -450.00?

A. PRIMARY KEY on order_id

B. DEFAULT 'PENDING'

C. CHECK (order_total >= 0.00)

D. NOT NULL on customer_id

QUESTION 6 · TRUE OR FALSE

An INSERT into orders that leaves out status stores 'PENDING', because of the DEFAULT clause.

A. True

B. False

Look back at the CREATE TABLE slide if you need it. Vote, then reveal.

QUIZ 2 · ANSWERS 4–6

Answers and explanations

QUESTION 4

Answer: B

O(N) to O(log N)

The index replaces a scan of every row with a tree search.

QUESTION 5

Answer: C

CHECK (order_total >= 0.00)

The CHECK constraint refuses any row whose total is negative. NOT NULL only blocks missing values.

QUESTION 6

Answer: A (True)

A DEFAULT fills in a value when the INSERT omits the column, so status becomes PENDING.

POSTGRESQL ECOSYSTEM · 0:35

We standardise on PostgreSQL 16

An enterprise-grade, open-source relational database.

Rich data types

- UUID
- JSONB
- Numeric

Extensible indexes

- B-Tree
- Hash
- GIN
- GiST

AI vector extensions

- `pgvector`

</> **Hands-on connection:** in Lab 01 you launch PostgreSQL in Docker Compose and benchmark flat files against an indexed query.

QUIZ 3 · QUESTIONS 1–3 · 0:40

Check your understanding

QUESTION 1

Which database system and version does this course standardise on?

A. MySQL 8

B. SQLite 3

C. PostgreSQL 16

D. MongoDB 7

QUESTION 2

Which index structures does PostgreSQL support, per the lecture?

A. Only B-Tree

B. B-Tree, Hash, GIN and GiST

C. Only Hash

D. None, it relies on flat files

QUESTION 3 · TRUE OR FALSE

The pgvector extension adds AI vector support to PostgreSQL.

A. True

B. False

Last lecture checkpoint: vote, then reveal.

QUIZ 3 · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: C

PostgreSQL 16

An enterprise-grade, open-source relational database, and the one every lab uses.

QUESTION 2

Answer: B

B-Tree, Hash, GIN and GiST

Extensible index structures are one of PostgreSQL's strengths. We start with B-Tree.

QUESTION 3

Answer: A (True)

pgvector is the AI vector extension named in the lecture.



BREAK · 0:45

Five-minute break

Back at 0:50 for Lab 01. Start Docker Desktop now and check that it works:

```
docker info
```

```
docker compose version
```

LAB 01 · 0:50

Environment bootstrap and File vs SQL benchmark

WARM-UP · 20 PTS

1. Container and healthcheck

Start PostgreSQL with Docker Compose and connect with psql.

Lab page: 15 min · class: 10 min

CORE · 50 PTS

2. DDL schema and data import

Create raw_orders and load 100,000 orders from CSV.

Lab page: 30 min · class: 20 min

BOSS LEVEL · 30 PTS

3. Index vs flat-file latency

Time a CSV scan against a B-Tree indexed query.

Lab page: 45 min · class: 20 min

100 points in total. Finish Exercise 3 at home if class time runs out.

LAB SETUP · 0:50 · 5 MIN

Docker Desktop and your project folder

1. Docker Desktop is running
2. docker-compose.yml is in your project folder
3. data/orders_100k.csv is in a data folder

CHECKPOINT

Raise a hand when both docker commands run without errors.

```
# Check your environment
docker info
docker compose version

# Expected project layout
kobokart/
  docker-compose.yml
  data/
    orders_100k.csv
```

The two project files are course materials: the lab page names them but does not include them. The layout is the one the lab commands assume.

EXERCISE 1 · WARM-UP · 0:55 · 10 MIN · 20 PTS

Container provisioning and healthcheck

```
# Start PostgreSQL service in detached mode
docker compose up -d postgres

# Connect to database shell
docker exec -it kobokart-db psql -U kobokart -d kobokart_db
```

PREDICT FIRST · 1 MIN

In the second command, which part is the container, which the user and which the database?

YOU ARE IN WHEN…

the prompt changes to the database name followed by =#

USEFUL IN PSQL

```
\l   list databases
\dt  list tables
\q   quit
```

EXERCISE 2A · CORE · 1:05 · 10 MIN · 50 PTS

DDL: create the raw_orders table

```
-- Create staging table for KoboKart order records
CREATE TABLE raw_orders (
    order_id UUID PRIMARY KEY,
    customer_id UUID NOT NULL,
    merchant_id UUID NOT NULL,
    order_total DECIMAL(10, 2) NOT NULL,
    status VARCHAR(20) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE
        DEFAULT CURRENT_TIMESTAMP
);
```

DATA TYPES MATTER

Use DECIMAL(10, 2) for currency, not float types, to avoid precision loss when aggregating order totals.

DECIMAL(10, 2)

10 digits in total, 2 after the decimal point. The largest value is 99,999,999.99.

Verify with \d raw_orders in psql.

EXERCISE 2B · CORE · 1:15 · 10 MIN

Import 100,000 orders with \copy

```
-- Copy CSV data into the staging table
\copy raw_orders(order_id, customer_id, merchant_id, order_total, status, created_at)
  FROM './data/orders_100k.csv' WITH (FORMAT csv, HEADER true);

-- Check the import
SELECT COUNT(*) FROM raw_orders;
```

WHAT IT DOES

HEADER true skips the CSV's first line. Columns are matched in the order listed.

SUCCESS CHECK

The count should be 100000, the size of the dataset.

IF “NO SUCH FILE”

The path is read where psql runs. Inside the container, the CSV must be visible there.

EXERCISE 3A · BOSS LEVEL · 1:25 · 10 MIN · 30 PTS

Create the index and profile the query

```
-- Create B-Tree index on status and order total
CREATE INDEX idx_raw_orders_status_total
  ON raw_orders (status, order_total DESC);

-- Profile query execution time
EXPLAIN ANALYZE
SELECT customer_id, SUM(order_total) AS total_spent
FROM raw_orders
WHERE status = 'PENDING' AND order_total > 250.00
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 10;
```

WHY THIS INDEX

The query filters on status (equality) and order_total (range). The index is sorted by both.

TRY IT

Run EXPLAIN ANALYZE before and after CREATE INDEX and compare.

EXERCISE 3B · BOSS LEVEL · 1:35 · 10 MIN

Time the flat file, then compare

```
# Suggested flat-file side (adjust to your CSV header)
import csv, time
from decimal import Decimal

t = time.perf_counter()
totals = {}
with open("data/orders_100k.csv", newline="") as f:
    for r in csv.DictReader(f):
        if r["status"] == "PENDING" and
Decimal(r["order_total"]) > Decimal("250.00"):
            c = r["customer_id"]
            totals[c] = totals.get(c, Decimal(0)) +
Decimal(r["order_total"])
top10 = sorted(totals.items(), key=lambda kv: kv[1],
reverse=True)[:10]
print(time.perf_counter() - t, "seconds")
```

READ THE EXPLAIN OUTPUT

Scan type: Seq Scan, Index Scan or Bitmap scan. Then the Execution Time line at the bottom, in ms.

Method	Time
CSV scan in Python	[__] ms
SQL, before index	[__] ms
SQL, with index	[__] ms

LAB QUIZ · QUESTIONS 1–3 · 1:45

Lab check: Docker and data types

QUESTION 1

Which command starts the PostgreSQL service in detached mode?

A. `docker exec -it kobokart-db psql`

B. `docker compose up -d postgres`

C. `docker info`

D. `psql -U kobokart`

QUESTION 2

In psql -U kobokart -d kobokart_db, what is kobokart_db?

A. The container name

B. The user name

C. The database name

D. The table name

QUESTION 3

Why use DECIMAL(10, 2) rather than a float type for order totals?

A. It is faster than integers

B. UUID columns require it

C. It uses less disk than float

D. Floats can lose precision when aggregating totals

Answer from what you just did in the lab, then reveal.

LAB QUIZ · ANSWERS 1–3

Answers and explanations

QUESTION 1

Answer: B

```
docker compose up -d postgres
```

The -d flag runs the service detached, in the background. Option A opens a shell in a running container.

QUESTION 2

Answer: C

The database name

-d selects the database. The container is kobokart-db with a hyphen, the user is kobokart.

QUESTION 3

Answer: D

Floats can lose precision

DECIMAL stores exact values, so sums of order totals stay accurate.

LAB QUIZ · QUESTIONS 4–6 · 1:49

Lab check: import and indexes

QUESTION 4

In \copy … WITH (FORMAT csv, HEADER true), what does HEADER true mean?

A. The first line is a header row, not loaded as data

B. A header row is added to the table

C. The file has no header

D. Column names are renamed

QUESTION 5

The index is on (status, order_total DESC). Which query does it serve best?

A. `WHERE customer_id = '…'`

B. `WHERE status = 'PENDING' AND order_total > 250.00`

C. `WHERE created_at < now()`

D. `No WHERE clause at all`

QUESTION 6 · TRUE OR FALSE

EXPLAIN ANALYZE only estimates the cost and never actually runs the query.

A. True

B. False

Vote, then reveal the answers on the next slide.

LAB QUIZ · ANSWERS 4–6

Answers and explanations

QUESTION 4

Answer: A

The first line is a header row

PostgreSQL skips it, so only the 100,000 data rows are loaded.

QUESTION 5

Answer: B

status = 'PENDING' AND order_total > 250.00

It filters on the index's leading column, then a range on the second. The other filters use columns the index does not cover.

QUESTION 6

Answer: B (False)

EXPLAIN ANALYZE executes the query and reports real run times next to the plan. Plain EXPLAIN only estimates.

WRAP-UP · 1:53

Scoring and evidence of your work

Exercise	Points
1 · Container and healthcheck	20
2 · DDL and data import	50
3 · Index vs flat-file benchmark	30
Total	100

SUGGESTED EVIDENCE TO KEEP

- ✓ Container running and the psql prompt
- ✓ \d raw_orders and the row count
- ✓ EXPLAIN ANALYZE output, with and without the index
- ✓ Your CSV timing and two sentences comparing it

RECAP · 1:56

What we covered today

1. Flat files fail under load: races, O(N) lookups, no integrity rules.
2. A DBMS adds declarative SQL, ACID transactions, WAL and indexes.
3. PostgreSQL 16 in Docker: container, psql, DDL and \copy.
4. An index plus EXPLAIN ANALYZE shows why SQL beats a file scan.

EXIT TICKET · 2 MIN

In one sentence: why does an indexed query beat scanning the CSV?

AFTER CLASS

Finish Exercise 3 if you ran out of time and keep your evidence.